ODS 2026/2 · VER-3 · EPI

Componente S3 — Alertas e Notificações

Encerramento de Sprint · reunião de quarta-feira com o coordenador · equipes transversais (VER-* e INV-1)

Apresentado por: Carlos Gabriel Teixeira

Squad VER-3 · EPI (líder: Ian Quadrio Raposo Branco Nunes)

Prova de conceito, cobertura de testes e dependências externas do componente S3.

01 · CONTEXTO

Onde o S3 está no pipeline do EPI

O VER-3 é um squad verticalizado: é dono da fatia inteira do tema EPI, da inferência até a aplicação.

I1	I9	S2	S3	A5
Detecção (MOD-1)	Uso correto? (VER-3)	Evento qualificado (LIB-1)	Alertas (VER-3 — aqui)	Aplicação EPI (VER-3)

Missão do S3: levar o evento à pessoa certa, no canal certo, e registrar o que aconteceu depois. É reutilizado por A1, A2, A3 e A7 — não tem nenhuma regra hard-coded de EPI.

02 · SPRINT 2

O que foi feito — motor funcional

Os dois itens pedidos para o S3 na Sprint 2 estão prontos e testados.

Ingestão de S2 (assinatura pub/sub)

- **contracts.py** — contrato interno do evento qualificado (EventoQualificado), com validação de campos obrigatórios e confiança entre 0 e 1.
- **ingestion.py** — adaptador que recebe JSON, valida e encaminha ao serviço.
- **subscription.py** — interface AssinanteEventos (contrato) + ConectorEventos, que liga um assinante de barramento ao ingestor sem depender da tecnologia escolhida.

Roteamento básico

- **routing.py** — RoteadorAlertas resolve severidade, destinatários e canais (SOM/TELA/PUSH) a partir do tipo de evento.
- **service.py** — AlertService orquestra: roteia, cria o alerta, evita duplicata por evento e salva.

02 · SPRINT 2

Como foi testado

56 testes automatizados (pytest), cobrindo cada módulo isoladamente e o fluxo de ponta a ponta.

Arquivo de teste	Cobre
`test_contracts.py`	Validação do evento qualificado
`test_alert.py / test_history.py`	Modelo do alerta e do registro de auditoria
`test_repository.py / test_repository_history.py`	Armazenamento, deduplicação, histórico
`test_routing.py`	Resolução de severidade/canal por tipo de evento
`test_service.py / test_service_routing.py`	Orquestração, idempotência de evento, reconhecer/silenciar
`test_ingestion.py`	JSON malformado, campo faltando, evento repetido
`test_subscription.py`	Assinante falso simulando o S2 publicando no barramento

Comando: `PYTHONPATH=src python -m pytest tests/s3 -q` — resultado: **56 passed**.

03 · PROVA DE CONCEITO

Demonstração ao vivo — roteiro

Simula o S2 publicando um evento, sem precisar do barramento real.

```
repo = AlertRepository()
roteador = RoteadorAlertas({"VIOLACAO_EPI": ConfiguracaoRoteamento(
    severidade="ALTA", destinatarios=("supervisor-01",), canais=("SOM","TELA"))})
service = AlertService(repository=repo, roteador=roteador)
ingestor = IngestorEventos(service)

# adaptador do B2 ainda não existe: usamos um assinante falso
assinante = AssinanteFalso()
ConectorEventos(assinante, ingestor).conectar()

# simula o S2 publicando um evento qualificado
alerta = assinante.publicar(
    '{"evento_id": "evt-900", "tipo": "VIOLACAO_EPI", "origem": "S2", '
    '"timestamp": "2026-10-08T10:00:00Z", "confianca": 0.9}'
)

print(alerta.severidade, alerta.destinatarios, alerta.canais, alerta.estado)
# -> ALTA ['supervisor-01'] ['SOM', 'TELA'] CRIADO

alerta2 = service.reconhecer_alerta(alerta.alerta_id, "supervisor-01")
print(alerta2.estado)  # -> RECONHECIDO
```

04 · SPRINT 3

O que foi feito — parcial

Ciclo de vida do alerta (reconhecimento e mudo) já está implementado.

Persistência e histórico (auditoria)

- **reconhecer_alerta(alerta_id, autor_id)** e **silenciar_alerta(alerta_id, autor_id)** em service.py.
- Cada ação gera um RegistroHistorico imutável (frozen dataclass) com autor e horário com fuso (timezone-aware).
- Um alerta já silenciado não pode ser reconhecido de novo.
- estado do alerta passa a refletir CRIADO → RECONHECIDO → SILENCIADO de verdade (antes ficava travado em CRIADO).

05 · PENDÊNCIAS

O que falta — por tipo de bloqueio

Pendência	Tipo	Observação
Persistência em banco real (hoje é dict em memória)	Interno	Decisão de infraestrutura ainda não tomada pelo squad.
Comandos repetidos de reconhecer/silenciar duplicam registro no histórico	Bug interno	Identificado nesta revisão; correção simples, não depende de ninguém.
Validação formal do timestamp e proveniência	Bloqueado por B3 (TRA-2)	B3 ainda não publicou o que é “horário oficial” nem os campos de proveniência.
Adaptador real de transporte (broker concreto)	Bloqueado por B2 (TRA-1)	TRA-1 ainda não definiu tecnologia, tópicos nem garantias de entrega.
Schema oficial do evento	Bloqueado por B1 (TRA-1)	contracts.py é um contrato provisório até B1 publicar o oficial.

06 · DESBLOQUEIO

O que fizemos para seguir sem esperar os outros squads

- **AssinanteEventos (Protocol) + ConectorEventos** — a fronteira com o B2 foi definida e testada contra um assinante falso. Quando o TRA-1 publicar o adaptador real, só trocamos a implementação — nada no S3 muda.
- **EventoQualificado como contrato provisório** — construído com os campos que o B1 já sinalizou como obrigatórios (origem, horário, confiança), pronto para ser ajustado assim que o schema oficial sair.
- **RoteadorAlertas desacoplado do tipo de aplicação** — a configuração de severidade/canal vem de fora (injeção de dependência), então o mesmo S3 atende A5 hoje e A1/A2/A3/A7 depois, sem reescrever nada.
- **Ciclo de vida do alerta já pronto** — mesmo sem banco real, reconhecer/silenciar já funcionam e já são auditáveis, adiantando trabalho da Sprint 3 que não dependia de infraestrutura.

Resultado: o S3 não fica parado esperando B1/B2/B3 responderem. Ele evolui com contratos provisórios e troca as peças quando os oficiais chegarem.

07 · PARA A REUNIÃO

Perguntas objetivas para TRA-1 e TRA-2

TRA-1 (B1 — contratos / B2 — transporte)

- Qual o schema oficial do evento que o S2 publica? Os nomes de campo batem com o que usamos?
- Qual tecnologia de barramento, nome do tópico e garantias de entrega (duplicata, ordem)?
- A fronteira S2→S3 passa pelo barramento, ou pode ser chamada direta?

TRA-2 (B3 — tempo e proveniência)

- Qual o formato oficial do campo de horário (ISO 8601? monotônico?)
- Quais campos de proveniência (câmera, versão do modelo) o S3 deve simplesmente repassar?

08 · RESUMO

Fechamento

Sprint 2 (motor funcional)	**COMPLETA**
Sprint 3 (auditoria e ciclo de vida)	**PARCIAL — persistência real e correção de bug pendentes**
Dependências externas (B1 / B2 / B3)	**MAPEADAS — não bloqueiam o avanço, só a versão final**

Próxima ação imediata: corrigir a duplicação de registro em reconhecer_alerta/silenciar_alerta antes da entrega de sexta-feira.